Simclr / Contrastive learning

Supervised learning : Requires millions of images that are labelled

Generative Models : learns to generate pixels (computationally expensive)

Pretext Tasks : Hand designed tasks that may limit generality

Goal :- To teach a model that two views of same image should have similar while different images should have different representations

The model learns invariance to transformations (crop, blur, color) while maintaining discrimination between images
↓
distinguishing

What is contrastive learning ?

It is a self supervised learning technique where model learns representations by comparing similar (positive) and dissimilar (negative) pairs of data points

Instead of using human-provided labels, the model creates its own supervision signal by asking - "Are these two images from same source or different sources?

Why SimCLR matters?

- No specialised architectures needed
- No memory banks required
- Simple augumentations + large batches can achieve 76.5%. Top-1 accuracy on Imagenet with linear evaluation, matching performance of supervised ResNet-50.

Positive Pairs

Image of a dog

↓

crop1 + color jitter

↓

view1

↓

crop2 + Blur

↓

view2

view1 ≈ view2

Negative pairs

view1 (dog)
view3 (cat)
view5 (car)
Dog ≠ cat ≠ car

Model learns invariance to transformations (crop, color, blur) while maintaining discrimination betn 2.

Architecture

original image

↓

1. Data augmentation → x_1, x_2 (Two views)

↓

2. Base encoder $f(\cdot)$ → h_1, h_2 (Representations)

↓

3. Projection head $g(\cdot)$ → z_1, z_2 (Projections)

↓

4. Contrastive Loss → Maximize $sim(z_1, z_2)$
Minimize $sim(z_1, z_others)$

After training we discard the projection head $g(\cdot)$ and use only encoder $f(\cdot)$

Component 1 Data Augmentation Module

Key augmentations used

1. Random crop + Resize

Extract different regions, resize to original dimensions

2. Color Jitter

Randomly adjust brightness, contrast, saturation, hue

3. Gaussian blur

Apply random blur to reduce texture bias

4. Horizontal flip

Random left-right flip.

Why this combination matters?

Crop + color distortion ensures that model doesn't cheat by matching colors instead of learning semantics.

Component 2

Architecture: ResNet 50 (or any CNN/ViT)

Function: Maps images to representation vectors (h)

These representation vectors h are what we actually use for downstream tasks after training.

Component 3 Projection head $g(\cdot)$ (The Adapter)

Architecture: Small MLP with one hidden layer.

$$z_i = g(h_i) = W^{(2)} * \sigma(W^{(1)} * h_i)$$

where $h_i \in \mathbb{R}^{2048}$ (i/p from encoder)

$z_i \in \mathbb{R}^{128}$ (o/p projection)

$W^{(1)} \in \mathbb{R}^{2048 \times 2048}$

$W^{(2)} \in \mathbb{R}^{2048 \times 128}$

σ = ReLU activation.

This adds non-linear projection betn representations and loss

Projection head $g(\cdot)$ creates a space z optimised for contrastive task.

After training we throw away $g(\cdot)$ and use h and $f(\cdot)$

Component 4 Contrastive Loss function (NT-xent)

Similarity matrix showing positive pairs and negative pairs.
Loss maximises similarity for positive pairs and minimizes for negative pairs

Temperature T (typically 0.1 to 0.5)

0.1 sharper distribution harder contrastive task, focuses on hard negatives

0.5 softer distribution, easier task

Tuning temperature is crucial for performance

For N original images

↓

Augumentation

↓

2N (views) (2 per image)

For each view 1 positive pair

2N-2 negative views

↓

cosine similarity betn positive and negative pairs.

we do contrastive learning on z instead of h because:-

h (encoder output): should be general purpose good for any downstream task.

z (projection o/p): optimised specifically for contrastive learning

Non linear adaptation :-

- ~~im~~ removes information that is harmful for downstream tasks
- Retains information that distinguish positive and negative pair.